Design Amazon Price Tracking Service

功能性需求（Functional Requirements）

1. 使用者應該能夠查看 Amazon 商品的價格歷史（透過網站或 Chrome extension）
2. 使用者應該能夠訂閱價格下跌通知，並設定通知門檻（透過網站或 Chrome extension）

不在範圍內（Out of scope）：

- 在平台上搜尋與探索商品
- 跨多個零售商的價格比較
- 商品評論與評分的整合

非功能性需求（Non-Functional Requirements）

1. 系統應優先考量可用性（Availability）而非一致性（Consistency）（可接受 eventual consistency）
2. 系統需能在規模化情境下處理 5 億（500 million）個 Amazon 商品
3. 系統需提供價格歷史查詢，延遲需低於 500ms
4. 系統需在價格變動後 1 小時內送出價格下跌通知

API 設計（API design）

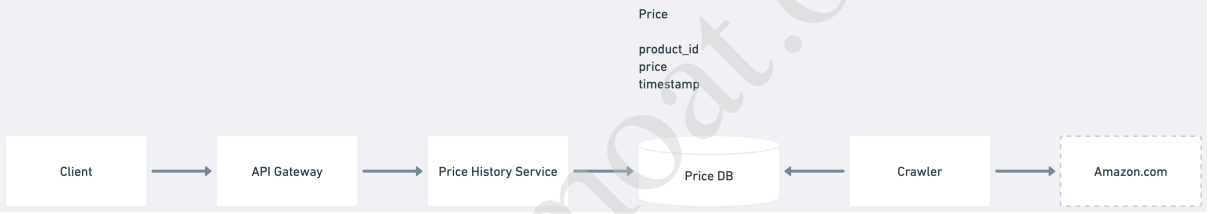
```
// Retrieve historical price data for charts and displays
GET /price/{product_id}?period=30d&granularity=daily -> PriceHistory[]
```

buildmoat.org

```
// Subscribe to price drop notifications with threshold
POST /subscriptions
{
  product_id,
  price_threshold,
  notification_type,
}
-> 200
```

High-Level Design

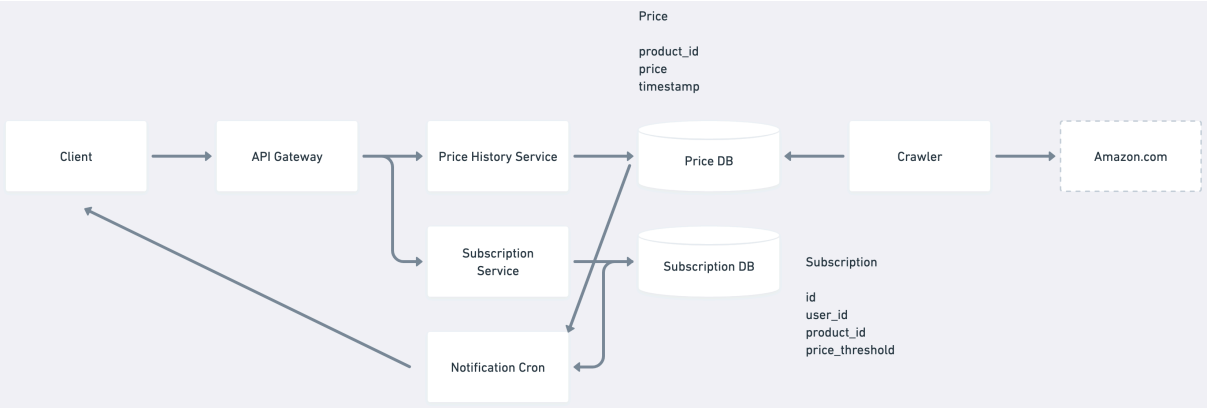
1. 使用者應該能夠查看 Amazon 商品的價格歷史（透過網站或 Chrome extension）



1. Client 向 Price History Service 發送 GET request
2. Price History Service 針對指定的 `product_id` 與時間範圍查詢 Price Table
3. Price History Service 回傳時間序列資料（time series data），可直接用於圖表（chart）渲染
4. 我們將 crawler 視為 black box，它會定期 crawl 並 scrape Amazon web site，並將價格資料寫入 DB。此部分之後會再深入探討
5. Price DB 需要具備非常高的吞吐量，且為 append-only（因為我們需要持久化商品的價格歷史）。DynamoDB 或 Cassandra 都是適合此存取模式的選擇
    - 為了提升查詢效率，我們可以使用 `product_id` 作為 partition key，timestamp 作為 sorting key

2. 使用者應該能夠訂閱價格下跌通知，並設定門檻（透過網站或 Chrome extension）

buildmoat.org



訂閱建立流程（Subscription creation flow）：

1. 使用者透過網站或 Chrome extension 提交訂閱
2. API Gateway 將 POST request 路由至 Subscription Service
3. Subscription Service 在 Subscriptions table 中建立一筆紀錄（user_id, product_id, price_threshold）
4. Service 回傳訂閱成功的確認給使用者

價格更新與使用者通知流程（Pricing update and user notification flow）：

1. Cron Job 每 2 小時執行一次以檢查是否需要發送通知
2. Job 查詢 Price table，找出最近 2 小時內的價格變動
3. 對於每一筆價格變動，從 Primary Database 查詢 Subscriptions table，找出 price_threshold ≥ new_price 的使用者
4. 對所有被觸發的訂閱發送 email 通知
5. 將通知標記為已送出，以避免重複發送

Subscription DB

我們會為使用者資料建立一個獨立的 Subscription DB table，因為它的寫入吞吐量低很多（僅來自使用者操作）。其存取模式為：當某個商品的價格更新到來時，我們需要查詢所有設定之 price_threshold ≤ 最新價格的使用者以進行通知。也就是：

```
SELECT user_id
FROM subscriptions
WHERE product_id = :product_id
AND price_threshold >= :new_price;
```

buildmoat.org

此使用情境下可以使用 Relational 或 NoSQL DB：

- 對於 SQL DB，我們可以將 (`product_id`, `user_id`) 設為 primary key，讓同一商品的所有訂閱資料聚集在一起；同時需要建立一個 secondary key（`product_id`, `price_threshold`, `user_id`）
- 對於 NoSQL DB，可以將 `product_id` 作為 partition key，並將 (`price_threshold`, `user_id`) 作為 sorting key

深入探討（Deep Dives）

1. 我們要如何有效率地發現並追蹤 5 億（500 million）個 Amazon 商品？

讓我們來看看該如何設計 web crawler 來爬取 Amazon.com。這裡有兩個主要挑戰需要解決：

1. 我們要如何發現 Amazon 上全部約 5 億個商品頁面？而且每天大約還會新增 3000 個新商品？
2. 我們要如何有效率地更新商品價格，並優先處理使用者最常查看的商品？

可行方案（Options）

1. **盲目爬取（Blindly crawling）所有 Amazon 頁面**

   本質上就是設計一個 web crawler，在 Amazon.com 網域內爬取所有頁面。基本步驟如下：

   1. 以少量 Amazon.com 頁面作為 seed
   2. 擷取這些頁面中的 links，並將它們放入 queue（breadth-first search）
   3. Crawler 以平行方式逐一處理 queue 中的頁面
   4. 追蹤已造訪的頁面以避免循環
   5. 在頁面解析時，將商品價格寫入 DB

   這個作法的問題在於，我們無法即時更新價格。由於 Amazon 對每個 IP 強制 rate limiting（1 visit/sec），即使我們有 1000 個不同的 IP，面對 5 億個商品頁面，仍然需要超過 5 天（5e8 / 1000 / 86400）才能完整掃過一次。這代表資料會過於陳舊（stale），實用性不高。

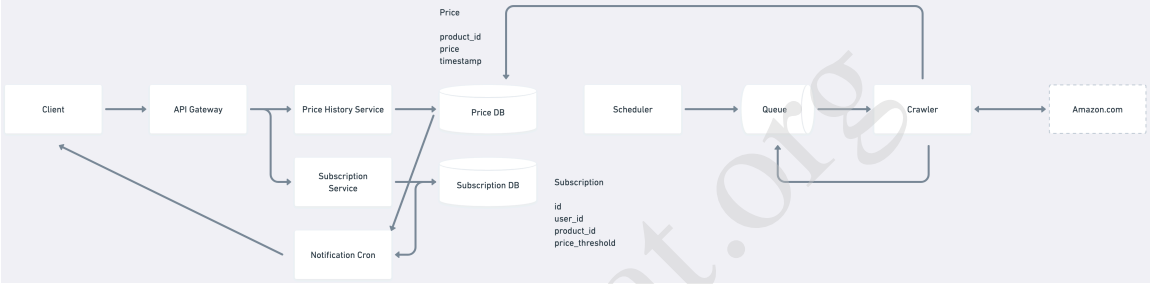
2. **優先式爬取（Prioritized Crawling）**

buildmoat.org

一個關於商品頁面的事實是：大部分的流量其實集中在一小部分商品上。我們可以利用這個特性，優先爬取使用者更有興趣的商品頁面，讓這些商品能更頻繁地更新價格。以下是一些可用來提升優先權分數的訊號：

1. 訂閱數較多的商品
2. 使用者搜尋次數較多的商品
3. 轉換率較高的商品（使用者從通知中點擊商品連結）

這個方法可以大幅提升使用者關注商品的價格更新頻率，但在「冷啟動（cold-start）」情境下仍可能不足，也就是：新的熱門商品如果尚未被使用者在我們的平台上搜尋，仍可能出現延遲。

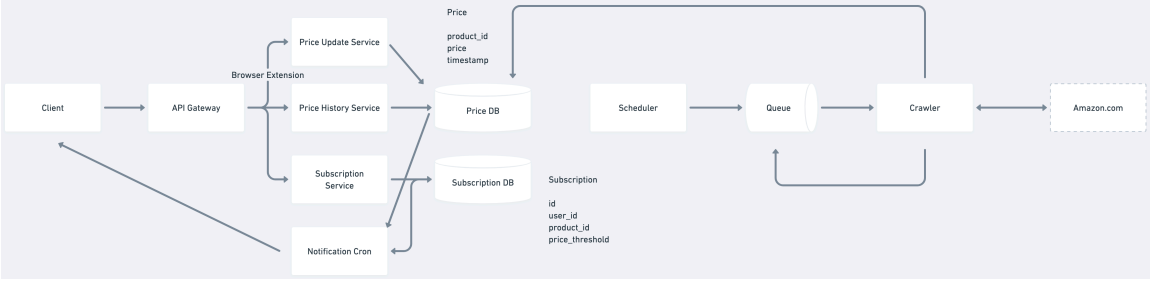


3. **善用 browser extension**

我們可以將 browser extension 視為一個分散式資料蒐集框架：

1. 當使用者安裝 extension 並瀏覽 Amazon 時，extension 會自動擷取 product IDs、即時價格與頁面 metadata，並回傳給後端服務
2. 我們可以即時收到使用者實際正在瀏覽商品的價格資料，這也自然地優先涵蓋了熱門與趨勢商品

這種使用者產生的資料（user-generated data）能涵蓋真正「有人在意」的商品，而不需要龐大的 crawler 基礎設施。我們的傳統 crawler 則只需要負責近期未被 extension 使用者查看過的商品。此外，extension 回傳的資料也能幫助我們發現新商品。



buildnoat.org

這種 hybrid approach 將我們最大的限制（必須監控數以百萬計的商品）轉化為競爭優勢，透過 crowdsourced data collection 來解決規模問題。與其用龐大的 crawler infrastructure 去對抗 Amazon 的 rate limits，我們改為利用使用者行為，自然地優先蒐集「最重要的商品」資料。

對於 user-generated data，我們必須注意惡意回報的風險。因此我們不會直接使用使用者上傳的資料，而是先驗證其完整性（integrity）。其中一個做法是：將使用者回報的更新優先送入 crawling system。如果系統偵測到可疑的價格變動（例如劇烈的價格下跌），就會以最高優先權重新爬取該商品頁面。如此一來，這些使用者回報的更新可以在數分鐘內被處理，並即時通知訂閱者。

2. 我們要如何有效率地處理價格變動，並通知已訂閱的使用者？

目前 notification service 的 high-level design 存在幾個問題：

1. 價格更新到使用者收到通知的延遲，高度依賴 cron job 的執行頻率
2. 每次 cron job 都需要進行昂貴的 full table scan

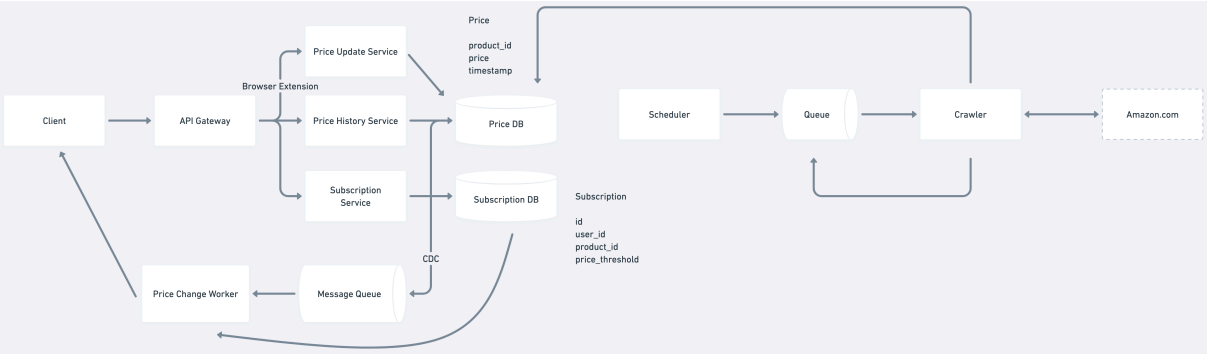
為了讓價格通知更即時，同時降低過度掃描資料表的成本，我們需要採用 event-driven approach。也就是：從「拉取（pull）」批次更新，轉為「推送（push）」單一事件。

其中一個做法是將 change data capture（CDC）與資料庫整合：

CDC = 把資料庫裡的變化「即時捕捉」出來，傳給其他系統使用。

- 當資料庫中有資料被 **新增 / 修改 / 刪除** 時，CDC 會將這些變化轉成事件
- 其他系統（例如 data warehouse、cache、search engine）就能即時收到更新，而不需要反覆掃描資料庫
- 常見實作方式：Log-based CDC
  - 多數資料庫（MySQL binlog、Postgres WAL、Oracle redo log）都有 transaction log
  - CDC 直接讀取這些 log，擷取所有變更事件，再轉成結構化訊息送往下游（例如 Kafka）

buildmoat.org



第二個選項是 **dual writes**：price collection service 在寫入資料庫的同時，也同步將事件發佈到 Kafka。當 crawler 或 extension 的更新進來時，負責寫入 Price DB 的 service 也會同時將結構化事件送到 notification stream。

Dual-write approach 讓我們能更聰明地判斷哪些價格變動需要觸發通知，例如過濾掉使用者不感興趣的小幅價格波動，或將短時間內的多次變動合併後再發佈事件。

Event-driven approach 的流程會變成：

1. 一個 process 持續 tailing database change logs，並將衍生事件發佈到 message queue
2. Message queue 的 consumer（例如 price change worker）消費事件後，查詢 Subscription DB 找出訂閱者（本質上是將 DB 與 event stream 做 join）
3. 若 `price_threshold` 高於最新價格，price change worker 就會向訂閱者發送通知

3. 我們要如何快速提供價格歷史查詢，以支援圖表生成？

當使用者查詢某商品的價格歷史時，查詢可能會像這樣：

```
SELECT
  date_trunc('day', "timestamp") AS day,
  avg(price) AS avg_price
FROM pricing_history
WHERE product_id = :pid
AND "timestamp" >= now() - interval '2 years'
GROUP BY 1
ORDER BY 1;
```

如果一個熱門商品每小時更新一次價格，單一商品在兩年內就會有 17,520 筆資料需要掃描。我們必須為數百萬使用者提供不同商品的價格圖表，而頻繁執行這類 analytics query，將無法在 DB 上滿足延遲需求。

buildmoat.org

可行方案（Options）

1. **每日預先彙總（Pre-aggregate）以優化讀取**

   透過排程 batch job，事先計算不同時間粒度的價格彙總：

   1. 每晚執行 cron job，計算所有商品的 daily、weekly、monthly 價格摘要。由於這是 heavy full table scan，我們可以額外啟用一個 read replica，專門用於夜間掃描
   2. 將結果存入單一的 `price_aggregations` table，並用一個 granularity 欄位標示該列資料是 daily、weekly 或 monthly
   3. 使用者請求價格圖表時，API 直接查詢 aggregation table，而不是 raw price data。例如：30 天圖表使用 daily aggregation，2 年圖表使用 monthly aggregation。在 (product_id, granularity, date) 上建立適當索引後，查詢可在毫秒內回傳數十筆預先計算好的資料

   ```
   PriceAggregation
    product_id
    avg_price
    min_price
    max_price
    granularity
    window
   ```

   Pre-aggregation 會帶來資料新鮮度（freshness）的問題，因為圖表資料最多可能落後 24 小時。不過對價格追蹤服務而言，使用者關注的是歷史趨勢而非即時價格，因此這種延遲通常可以接受。此方法也需要額外的儲存空間來保存預先計算的摘要資料，但其規模會隨著商品數量與支援的時間區間線性成長。

2. **使用 OLAP / time-series DB 儲存歷史資料**

   如果我們還有其他類型的分析需求（例如跨商品查詢、heavy joins），可以考慮將資料導入 TSDB（如 InfluxDB）或 OLAP DB（如 ClickHouse）。相較於 row-based 的 OLTP DB，這類 DB 多為 column-oriented storage，能非常有效率地執行 aggregation query（sum、avg 等）。

   常見的 ingestion pattern 會透過 CDC，資料流為：

   OLTP → CDC → stream / connector → TSDB / OLAP

   這種做法能在維持即時回應能力的同時，提供生產環境等級的效能與彈性，用於大規模的價格圖表服務。

buildmoat.org